

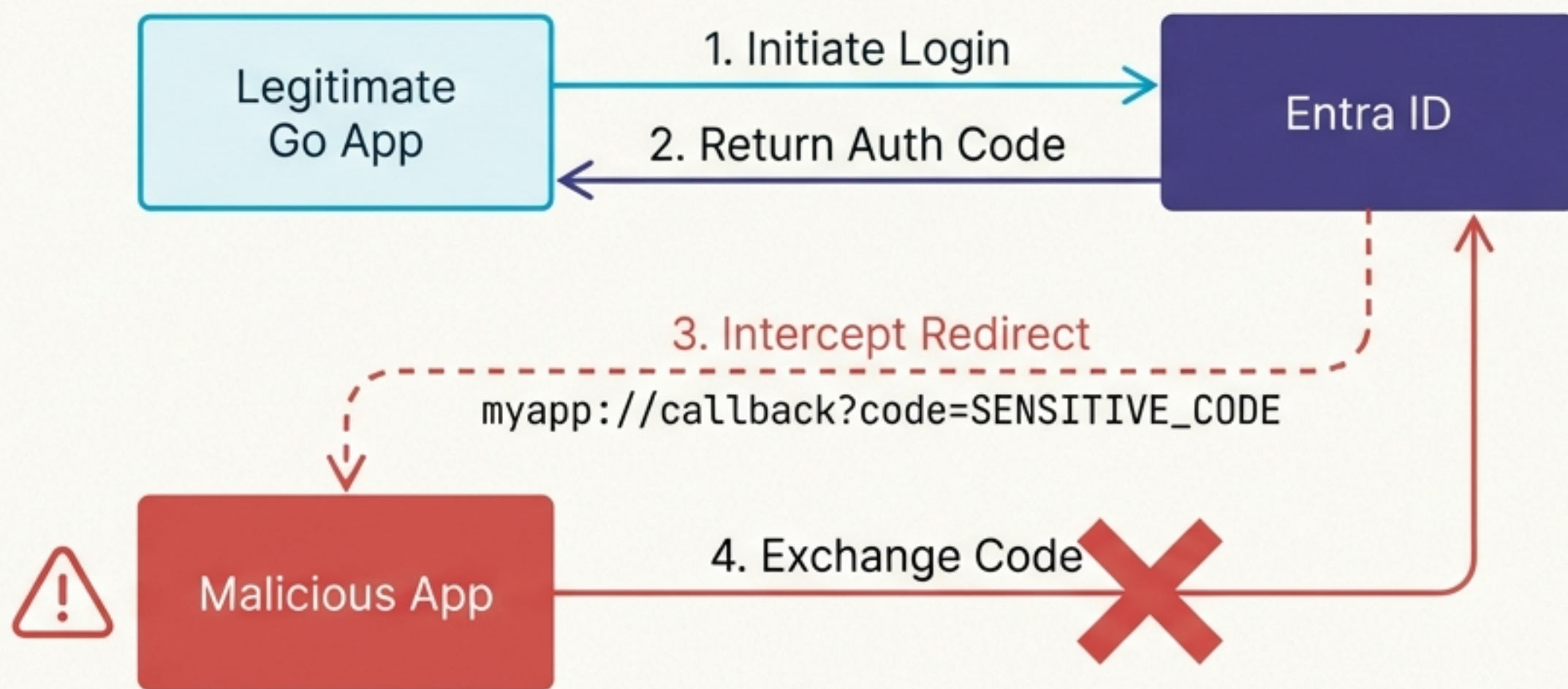


From Theory to Terminal: Securing Go Applications with Entra ID and PKCE

A definitive guide to building secure, modern clients and APIs using industry-best practices.

Modern Apps Have a Hidden Vulnerability

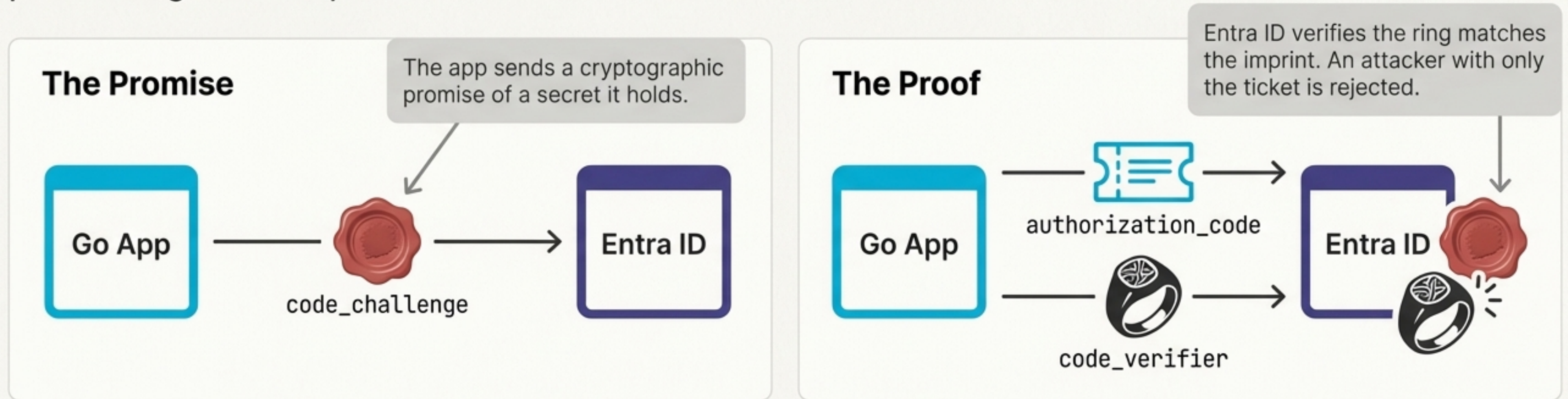
The standard OAuth 2.0 authorization code flow has a critical weakness for public clients like desktop and CLI apps: the **Authorization Code Interception Attack**.



Because public clients (desktop, mobile, CLI) cannot securely store a client secret, an intercepted authorization code is all an attacker needs to impersonate your user.

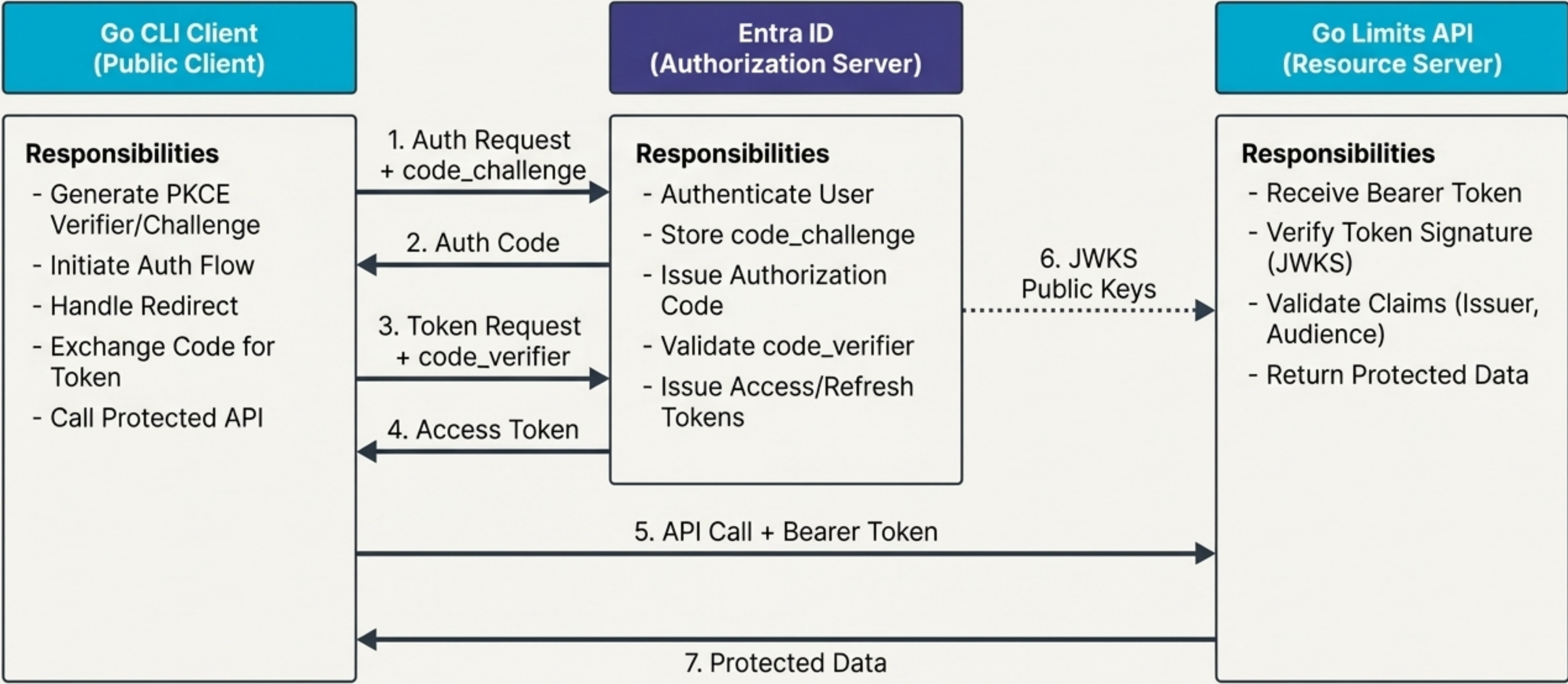
The Solution: Proof Key for Code Exchange (PKCE)

PKCE (RFC 7636) defeats the interception attack by cryptographically binding the token exchange to the initial authorization request. It proves that the same application instance is performing both steps.



1. Client creates a secret **code_verifier**.
2. Client creates a public **code_challenge** by SHA256 hashing the verifier.
3. Client sends the **code_challenge** to start the login.
4. Client sends the **code_verifier** to exchange the code for a token.
5. Server validates: $\text{SHA256}(\text{received_verifier}) == \text{stored_challenge}$

The Architectural Blueprint



Implementing the Client: Generating the Cryptographic Proof

The entire security of PKCE begins with generating a high-entropy secret (code_verifier) and its corresponding public proof (code_challenge) for each authentication request.

```
// generateCodeVerifier creates a high-entropy cryptographic random string.
func generateCodeVerifier() string {
    b := make([]byte, 32)
    rand.Read(b)
    return base64.RawURLEncoding.EncodeToString(b)
}
```

32 bytes = 256 bits of entropy

URL-safe encoding is required.

```
// generateCodeChallenge creates a SHA256 hash of the verifier.
func generateCodeChallenge(verifier string) string {
    hash := sha256.Sum256([]byte(verifier))
    return base64.RawURLEncoding.EncodeToString(hash[:])
}
```

One-way hash. Verifier cannot be derived from this.

This secret (code_verifier) is ephemeral. It exists only in memory for the duration of a single login flow and is **never** stored, eliminating the need for client-secret management.

Implementing the Client: Exchanging the Code for a Token

After the user authenticates, the client receives a temporary authorization code. It then makes a direct, back-channel request to the token endpoint, presenting the original **code_verifier** as proof.

```
func (am *AuthManager) exchangeCodeForToken(code string) error {
    tokenURL := fmt.Sprintf(tokenEndpoint, tenantID)
    data := url.Values{
        "client_id":    {clientID},
        "grant_type":   {"authorization_code"},
        "code":         {code},
        "redirect_uri": {redirectURI},
        "code_verifier": {am.codeVerifier},
    }
    resp, err := http.PostForm(tokenURL, data)
    // ... handle response and parse tokens
}
```

The temporary code from the redirect.

CRITICAL: The original secret proves we initiated the flow.

The code_verifier is the key. An attacker who only intercepted the code cannot complete this step, and the request will be rejected by Entra ID.

What If There Is No Browser?

The PKCE flow is ideal for apps on a user's desktop, but what about headless environments like SSH sessions, containers, or remote servers?

Solution: The Device Code Flow

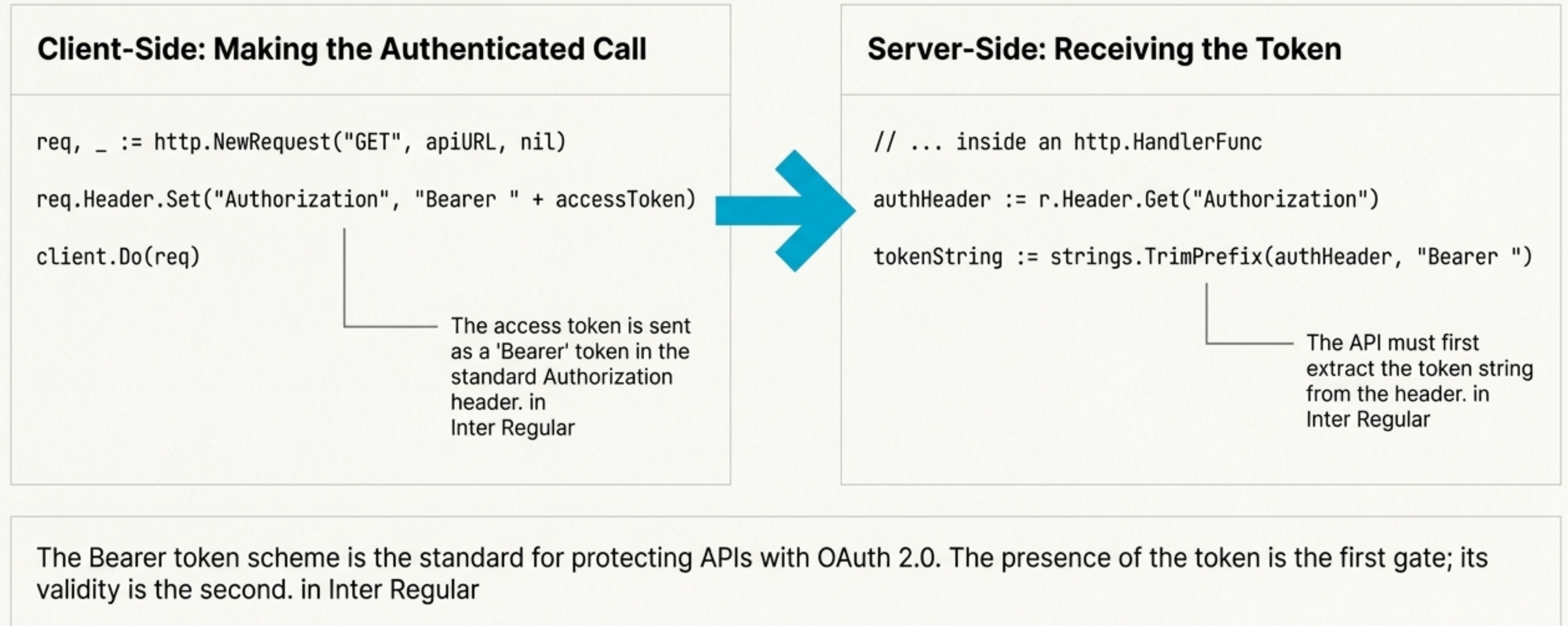
The industry-standard solution for CLI and headless tools (used by Azure CLI, GitHub CLI, kubectl).



- ✓ **Works without** a local browser.
- ✓ Fully supports MFA and Conditional Access policies.
- ✓ Far more secure than discouraged alternatives like the Username/Password flow.

Securing the API: Authenticating Requests

Once the client has an access token, it must present it with every API request. The API's primary responsibility is to validate this token before processing the request.



Level 2 Security: You Must Validate the Claims

A token that looks like a token is not enough. Your API must validate its claims to ensure it was issued for the correct purpose, to the correct audience, and is not expired.

Claim	Example Value	Why It's Critical
`iss` (Issuer)	https://login.microsoftonline.com/{tenantID}/v2.0	Confirms the token was issued by your trusted authority (your Entra ID tenant). Prevents accepting tokens from other tenants or providers.
`aud` (Audience)	{your_client_id}	Confirms the token was intended for your specific application. Prevents a token issued for another app from being used to access your API (a "confused deputy" attack).
`exp` (Expiration Time)	`1672531199`	Prevents replay attacks. Ensures the token is only valid for its intended, short lifetime.
`nbf` (Not Before)	`1672527600`	Ensures the token is not used before it is considered valid.

These checks are performed after parsing the token, as seen in the `validateEntraIDToken` function.

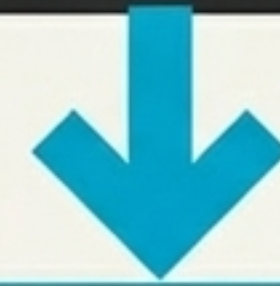
Production-Grade Security: Verifying the Signature

- **The Problem:** Validating claims is good, but how do you know the token itself wasn't forged? An attacker could create a fake token with valid-looking claims.
- **The Solution:** Cryptographic Signature Verification: The token is digitally signed by Entra ID using a private key. Your API must verify this signature using Entra ID's corresponding public key. This mathematically proves two things:
 1. **Authenticity:** The token was genuinely issued by Entra ID.
 2. **Integrity:** The token's payload (claims) has not been tampered with since it was issued.
- **The Mechanism:** JSON Web Key Set (JWKS): Entra ID publishes its public keys at a well-known JWKS endpoint. Your API must fetch these keys to perform the signature validation.

[eyJhbGciOiJSUzI1Ni...] .

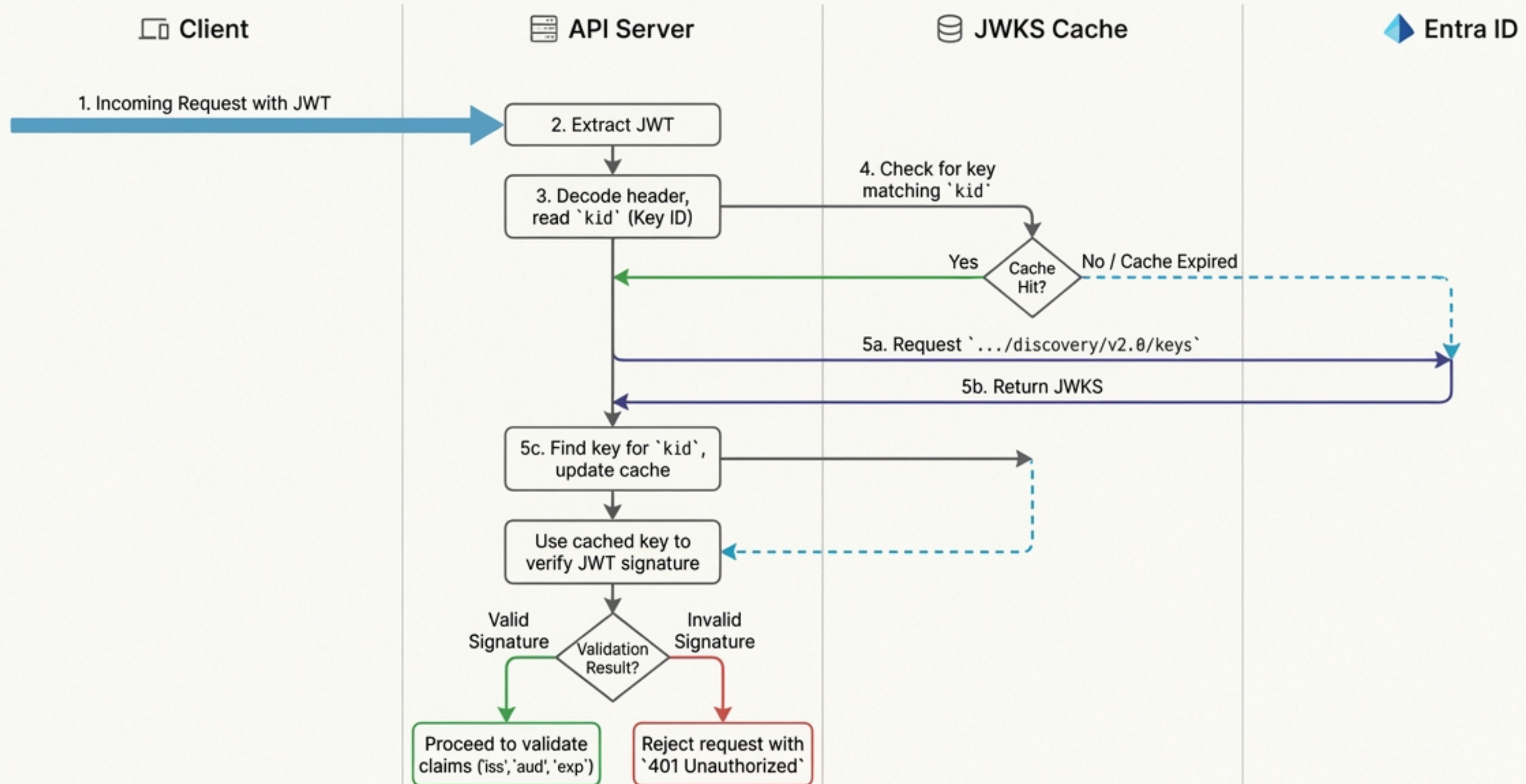
[eyJpc3MiOiJodHRwcz...] .

[Gt8Ql5yZq9w3a...]



This part proves the Header and Payload are authentic and untampered.

How JWKS Signature Validation Works



The JWKS Cache in Action: A Resilient Implementation

Constantly fetching the JWKS endpoint for every request would be slow and inefficient. A thread-safe, time-to-live (TTL) cache is the correct production pattern.

The Cache Structure

```
// JWKSCache caches the public keys
type JWKSCache struct {
    keys      map[string]*rsa.PublicKey
    lastFetch time.Time
    mu        sync.RWMutex
    ttl       time.Duration
}
```

In-memory map from Key ID (kid) to the parsed RSA public key.

Mutex to ensure thread-safe access.

e.g., 24 hours. Keys are refreshed periodically.

The Core Logic in `getKey()`

```
// ...
if exists && !needsRefresh {
    return key, nil
}
// Refresh cache if needed
if needsRefresh || !exists {
    if err := c.refresh(); err != nil {
        if exists { return key, nil }
        return nil, err
    }
}
// ...
```

The logic automatically handles cache misses and TTL expiration.

If refresh fails, fall back to stale cache to maintain availability.

Benefits of This Approach

- **Performance:** Avoids network latency on most requests.
- **Resilience:** Handles Entra ID key rotation automatically.
- **Availability:** Can function with a stale cache if the JWKS endpoint is temporarily unavailable.

An Extra Layer of Assurance: Validating with Microsoft Graph

After cryptographically verifying the token's signature and claims (offline validation), you can perform an optional online check to ensure the token hasn't been revoked and the user session is still active.

The Technique

Use the received access token to make a simple call to a well-known Microsoft Graph endpoint, like /me. If the call succeeds (200 OK), the token is confirmed to be active and valid. If the call fails (401 Unauthorized), it could indicate the session was revoked, the user changed their password, etc.

```
func validateWithMicrosoftGraph(accessToken string) (*UserInfo, error) {  
    req, _ := http.NewRequestWithContext(ctx, "GET", graphAPIURL, nil)  
  
    req.Header.Set("Authorization", "Bearer " + accessToken)  
  
    client := &http.Client{}  
    resp, err := client.Do(req)  
    // ...  
    if resp.StatusCode != http.StatusOK {  
        return nil, fmt.Errorf("Microsoft Graph returned status %d", resp.StatusCode)  
    }  
    // ...  
}
```

Pass the user's token directly to the Graph API.

A non-200 status from Graph is a strong signal the token is invalid.

This provides defense-in-depth, trading a small amount of latency for a higher degree of confidence in the token's validity.

The Complete Flow: From Client Request to API Response

Step	Action	Participants	Security Principle
1	Generate Verifier & Challenge	Go Client	Ephemeral, per-request secrets
2	Redirect User to Login (with Challenge)	Client → Browser → Entra ID	Binds the session to a cryptographic proof
3	Redirect with Auth Code	Entra ID → Browser → Client	Secure code delivery via browser redirect
4	Exchange Code + Verifier for Tokens	Client → Entra ID	PKCE Verification: Proves client identity
5	Call API with Bearer Token	Go Client → Go API	Standardized API authentication
6	Validate Signature (JWKS) & Claims	Go API → Entra ID (Keys)	Token Validation: Ensures authenticity & integrity
7	Return Protected Data	Go API → Go Client	Authorization successful

The Modern Standard for Secure Application Development

- **For Public Clients, PKCE is Non-Negotiable.**
 - It is the industry standard for securing desktop, mobile, CLI, and Single-Page Applications. It eliminates the need for unsecurable client secrets and prevents authorization code interception.
- **For APIs, Signature Validation is Mandatory.**
 - Never trust a JWT without cryptographically verifying its signature using the issuer's public JWKS keys. This is your only guarantee against token forgery.
- **Implement a Resilient JWKS Cache.**
 - A smart cache provides performance, availability, and automatically handles provider key rotation, making your validation robust.
- **Always Validate Claims.**
 - Verifying the issuer (`iss`) and audience (`aud`) is just as important as verifying the signature to prevent token misuse.

By combining a PKCE-enabled client with a JWKS-aware API, you build a system with defense-in-depth, protecting your users and your data according to modern security best practices.